Lab 03

COMP1322 Programming II

Student Name: Qi Liu
Student ID: 36557854
Student Email: ql1e24@soton.ac.uk

Date: March 5, 2026

1: Java Basics Recap

1.1. Class & Object Creation

Code:

```java
class Person {  4 个用法

    String name; // stores the name  2 个用法
    int age; // stores the age  2 个用法

    Person(String name, int age) {  2 个用法
        this.name = name; // "this.name" means the class variable, "name" is what we passed in
        this.age = age;
    }

    // Printing person's info
    void displayInfo() {  2 个用法
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

public class Task1_1_Person {
    public static void main(String[] args) {
        Person person1 = new Person( name: "Max",  age: 20); // create the first Person object

        Person person2 = new Person( name: "Lando",  age: 25); // create the second one

        person1.displayInfo(); // call displayInfo() on each person to print their details
        System.out.println("---");
        person2.displayInfo();
    }
}
```

Output:

```
Name: Max
Age: 20
---
Name: Lando
Age: 25
```

Describe what an object is and how it is created in Java.

An object is created from a class, which contains attributes and methods. And attributes work like variables which are used for storing data. Then method is kind of like a function, it can define actions that an object can perform.

And the method to create an object is to declare a reference variable at the beginning, then create the object using **new** keyword. In the end we assign it to the reference so that we create an object successfully.

1.2. Variable Types & Scope

Code:

```java
import java.util.ArrayList; // ArrayList

public class Task1_2_PassByValue {
    static void tryToChangeNumber(int num) {  1个用法
        num = 999; // change the copy, not the original
        System.out.println("Inside method, num = " + num);
    }

    static void addToList(ArrayList<Integer> list) {  1个用法
        list.add(999); // adds 999 to the actual list
        System.out.println("Inside method, list = " + list);
    }

    public static void main(String[] args) {
        int myNumber = 5; // create an int number
        System.out.println("Before method call, my number is: " + myNumber);
        tryToChangeNumber(myNumber); // try to change it
        System.out.println("After method call, my number is: " + myNumber);

        System.out.println("---");

        // Pass by Reference example
        ArrayList<Integer> myList = new ArrayList<>(); // create a list
        // add numbers
        myList.add(1);
        myList.add(2);
        myList.add(3);
        System.out.println("Before method call, my list is: " + myList); // [1, 2, 3]
        addToList(myList); // add 999
        System.out.println("After method call, my list is: " + myList); // [1, 2, 3, 999]
    }
}
```

Output:

```
Before method call, my number is: 5
Inside method, num = 999
After method call, my number is: 5
---
Before method call, my list is: [1, 2, 3]
Inside method, list = [1, 2, 3, 999]
After method call, my list is: [1, 2, 3, 999]
```

Explain the difference between pass-by-value and pass-by-reference with examples.
pass-by-value is created for copying the variable's value and then passing it to the method that we are calling next. And whatever any kind of changes inside the method will not affect the original variable.
Pass-by-reference means giving a function arguments that give the function a reference to the original variable instead of a copy of its value.

2: Methods & Overloading

2.1. Method Overloading Practice

Code:

```java
public class Task2_1_Calculator {

    int add(int a, int b) {  1个用法
        return a + b; // just add them and return the result
    }

    double add(double a, double b) {  1个用法
        return a + b; // add them and return the result
    }

    String add(String a, String b) {  1个用法
        return a + b; // joins strings together
    }

    public static void main(String[] args) {
        Task2_1_Calculator calc = new Task2_1_Calculator(); // create a Calculator object

        int result1 = calc.add( a: 6,  b: 7); // call the int version
        System.out.println("int add: " + result1);

        double result2 = calc.add( a: 6.6,  b: 7.7); // call the double version
        System.out.println("double add: " + result2);

        String result3 = calc.add( a: "Hello!",  b: " I am a Year 1 student at the University of Southampton"); // call the String version
        System.out.println("String add: " + result3);
    }
}
```

Output:

```
int add: 13
double add: 14.3
String add: Hello! I am a Year 1 student at the University of Southampton
```

What is method overloading? Why is it useful?
Method overloading means defining multiple methods with the same name in the same class. But they are under different parameter lists.
Why it is useful is that it uses the same method name for similar operations so that we only need to remember one method name. And the compiler determines which method to execute during compilation.

2.2. Calling Static & Non-Static Methods

Code:

```java
public class Task2_2_Greeting {
    static void sayHello() {  1个用法
        System.out.println("Hello, World!"); // static method can call it without creating an object
    }

    void personalizedGreeting(String name) {  2个用法
        System.out.println("Hello, " + name + "!"); // non-static method need to create an object first
    }

    public static void main(String[] args) {
        Task2_2_Greeting.sayHello(); // call the static method

        Task2_2_Greeting greet = new Task2_2_Greeting(); // create the object
        greet.personalizedGreeting( name: "Alice"); // now we can call the method
        greet.personalizedGreeting( name: "Bob");   // call it again with a different name
    }
}
```

Explain the difference between static and non-static methods with examples.
A static method belongs to the class instead of an object. It can be called without creating an object and shared by all objects of the class.

Non-static methods belong to object. And before we use it we have to create an object first and every object can use the method.

3: Encapsulation & Java Library Usage

3.1. Encapsulation

Code:

```java
public class Task3_1_BankAccount {
    private String accountNumber; // stores account number  2 个用法
    private double balance; // stores money amount  5 个用法

    Task3_1_BankAccount(String accountNumber, double startingBalance) {  1 个用法
        this.accountNumber = accountNumber; // set account number
        this.balance = startingBalance; // set starting balance
    }

    double getBalance() {  1 个用法
        return balance; // return current balance
    }

    void deposit(double amount) {  1 个用法
        if (amount > 0) { // check the amount is positive or not
            balance = balance + amount; // add to balance
            System.out.println("Deposited: " + amount);
        } else {
            System.out.println("Invalid deposit amount!"); // reject negative input
        }
    }

    // shows the account info
    void displayInfo() {  1 个用法
        System.out.println("Account Number: " + accountNumber); // print account number
        System.out.println("Balance: " + balance); // print balance
    }

    public static void main(String[] args) {
        // Create a bank account object
        Task3_1_BankAccount account = new Task3_1_BankAccount( accountNumber: "Account-001",  startingBalance: 667.7);

        account.displayInfo(); // show starting info

        account.deposit( amount: 7766.0); // deposit some money

        System.out.println("New balance: " + account.getBalance()); // check balance using getter
    }
}
```

Output:

```
Account Number: Account-001
Balance: 667.7
Deposited: 7766.0
New balance: 8433.7
```

Why is encapsulation important? How does it improve security in programming?
Encapsulation can make sure that data won't be changed directly. In a real developing environment, variables are usually made private, and methods like getters and setters are used to access them.
Since we can not change the data directly, other classes cannot change the variable directly. And methods can check if the data is valid before program saving it.

3.2. Using Java Collections

Code:

```java
import java.util.ArrayList;
import java.util.Iterator;

public class Task3_2_StudentList {

    public static void main(String[] args) {
        ArrayList<String> students = new ArrayList<>(); // create an ArrayList to store student names

        // add student names to the list
        students.add("Max");
        students.add("Lando");
        students.add("Charlies");
        students.add("Lewis");

        System.out.println("Student Names:");

        Iterator<String> iterator = students.iterator(); // get the iterator one by one to go through the list

        // checks if there is another item left in the list
        while (iterator.hasNext()) {
            String name = iterator.next(); // next() gives us the next item
            System.out.println(name);
        }
    }
}
```

Output:

```
Student Names:
Max
Lando
Charlies
Lewis
```

What is an Iterator, and why is it useful for collections in Java?
An Iterator allows me to loop through elements in a collection at a time but we need to import java.util.Iterator first.
It is useful because it is easy to traverse collections and allows elements to be removed during iteration.

4: Inheritance & Polymorphism

4.1. Implementing Inheritance

Code:

```java
import java.util.ArrayList;

// superclass (parent class)
class Animal {  4 个用法  2 个继承者
    String name;  4 个用法

    // constructor for Animal
    Animal(String name) {  2 个用法
        this.name = name; // set the name
    }

    // default sound method
    void makeSound() {  1 个用法  2 个重写
        System.out.println(name + "Animal makes a sound"); // default message
    }
}

// subclass of animal (child class) dog
class Dog extends Animal {  2 个用法
    Dog(String name) { // constructor for dog  2 个用法
        super(name); // calls Animal's constructor to set the name
    }

    @Override // replacing the parent's makeSound() method  1 个用法
    void makeSound() {
        System.out.println(name + ": Bark!");
    }
}

// subclass of Animal cat
class Cat extends Animal {  2 个用法
    Cat(String name) { // constructor for cat  2 个用法
        super(name); // call parent constructor
    }

    @Override  1 个用法
    void makeSound() {
        System.out.println(name + ": Meow!");
    }
}
```

```java
public class Task4_1_Animals {

    public static void main(String[] args) {
        ArrayList<Animal> animals = new ArrayList<>(); // holds Animal objects

        animals.add(new Dog( name: "Ab"));
        animals.add(new Cat( name: "Bob"));
        animals.add(new Dog( name: "Buddy"));
        animals.add(new Cat( name: "Charlie"));

        for (Animal a : animals) {
            a.makeSound(); // calls Dog or Cat version depending on actual type
        }
    }
}
```

Output:

```
Ab: Bark!
Bob: Meow!
Buddy: Bark!
Charlie: Meow!
```

Why does IntelliJ prompt you to add @Override?
Because the annotation @Override is a method that overrides a method in the superclass or interface. And it can help programs to detect potential errors and make the code easier to read.

What happens if you don't add it? Will the program throw an error?
Even though we don't add it, the code still can run successfully, but it may not detect other possible errors, such as the wrong method names.

What is the actual purpose of @Override in Java?
The actual purpose is used for error checking and code/method clarity.

4.2. Method Overriding & Superclass Reference

Code:

```java
class Animal2 {  2 个用法  1 个继承者
    String name;  4 个用法

    Animal2(String name) {  1 个用法
        this.name = name;
    }

    void makeSound() {  1 个用法  1 个重写
        System.out.println(name + ": Animal makes a sound");
    }

    // added to superclass
    void sleep() {  1 个用法
        System.out.println(name + " is sleeping... Zzz");
    }
}

class Dog2 extends Animal2 {  1 个用法
    Dog2(String name) {  1 个用法
        super(name); // call parent constructor
    }

    @Override  1 个用法
    void makeSound() {
        System.out.println(name + ": Bark!");
    }
}

public class Task4_2_DynamicDispatch {

    public static void main(String[] args) {
        Animal2 a = new Dog2( name: "Max"); // create a Dog2 object but reference it as an Animal2

        a.makeSound(); // calls Dog2
        a.sleep();     // calls Animal2's sleep() because Dog2 doesnt override it
    }
}
```

Output:

```
Max: Bark!
Max is sleeping... Zzz
```

How does dynamic method dispatch work in Java?

It occurs when a reference to the superclass points to an object of the subclass that has overridden the method.

5: Advanced Java Library - HashMaps

5.1. Storing Key-Value Pairs

Code:

```java
import java.util.HashMap; // HashMap

public class Task5_1_HashMap {

    public static void main(String[] args) {
        HashMap<String, Integer> ages = new HashMap<>();  // create a HashMap to stores pairs of (key, value)

        // Add key and value
        ages.put("Max", 20);
        ages.put("Lando", 25);
        ages.put("Charlies", 22);

        // Retrieve a value using the key
        int maxAge = ages.get("Max"); // get Max's age
        System.out.println("Max's age: " + maxAge);

        System.out.println("All entries:");
        for (String name : ages.keySet()) { // gives us all keys
            System.out.println(name + " is " + ages.get(name));
        }
    }
}
```

Output:

```
Max's age: 20
All entries:
Max is 20
Charlies is 22
Lando is 25
```

How does a HashMap work in Java? What are its advantages?

HashMap can store data in pairs of keys and values and uses a hash function to determine the bucket for each key.

The advantages of a HashMap:

Keys are unique, and values can be duplicated.

Null key (only one) and multiple null values are allowed.

Uses hashing for fast access.

Linked lists are used for avoiding collisions.

5.2. Using Generics Properly

Code:

```java
import java.util.ArrayList; // need ArrayList

public class Task5_2_Generics {

    public static void main(String[] args) {
        ArrayList listWithoutGenerics = new ArrayList(); // no type specified
        listWithoutGenerics.add("Hello"); // add a String
        listWithoutGenerics.add(123); // add an Integer
        listWithoutGenerics.add(3.14); // add a Double

        System.out.println("Without generics list: " + listWithoutGenerics);

        try {
            String s = (String) listWithoutGenerics.get(1); // get(1) is 123 (an Integer) but I try to cast it to String
        } catch (ClassCastException e) {
            System.out.println("Runtime error: " + e.getMessage()); // catch the crash
        }

        System.out.println("---");

        ArrayList<String> listWithGenerics = new ArrayList<>(); // only Strings allowed
        listWithGenerics.add("Hello");
        listWithGenerics.add("World");

        System.out.println("With generics list: " + listWithGenerics);

        // No casting needed due to it is always a String
        String s = listWithGenerics.get(0);
        System.out.println("First item: " + s);
    }
}
```

Output:

```
Without generics list: [Hello, 123, 3.14]
Runtime error: class java.lang.Integer cannot be cast to class java.lang.String (java.lang.Integer and java.lang.String are in module java.base of loader 'bootstrap')
---
With generics list: [Hello, World]
First item: Hello
```

What are Generics, and why should we use them in Java collections?
In a collection, generics avoid the addition of wrong types of objects, eliminating the need for casting.

2: Library Management System

Code:

```java
import java.util.ArrayList;
import java.util.Scanner;

// superclass Publication for all types of publications
class Publication {  4 个用法  2 个继承者

    // private makes only this class can access them
    private String title;  3 个用法
    private String author;  3 个用法
    private int publicationYear;  3 个用法

    // constructor to set up the publication
    Publication(String title, String author, int publicationYear) {  2 个用法
        this.title = title;
        this.author = author;
        this.publicationYear = publicationYear;
    }

    // get title
    String getTitle() {  0 个用法
        return title;
    }

    // get author
    String getAuthor() {  0 个用法
        return author;
    }

    // get publicationYear
    int getPublicationYear() {  0 个用法
        return publicationYear;
    }

    // display publication info
    void displayInfo() {  3 个用法  2 个重写
        System.out.println("Title: " + title);
        System.out.println("Author: " + author);
        System.out.println("Publication Year: " + publicationYear);
    }
}

```

```java
// child class of publication - Book
class Book extends Publication {  2 个用法

    private String ISBN; // unique ID for the book  3 个用法

    // constructor
    Book(String title, String author, int publicationYear, String ISBN) {  1 个用法
        super(title, author, publicationYear); // call publication constructor
        this.ISBN = ISBN; // set the ISBN
    }

    // get ISBN
    String getISBN() {  0 个用法
        return ISBN;
    }

    @Override // Override displayInfo() to also show the ISBN  3 个用法
    void displayInfo() {
        super.displayInfo(); // call the parent method to print title, author, year
        System.out.println("ISBN: " + ISBN); // add ISBN on top
    }
}

class Magazine extends Publication {  2 个用法

    private int issueNumber; // issue of magazine  3 个用法

    Magazine(String title, String author, int publicationYear, int issueNumber) {  1 个用
        super(title, author, publicationYear); // call publication constructor
        this.issueNumber = issueNumber; // set the issue number
    }

    // get issueNumber
    int getIssueNumber() {  0 个用法
        return issueNumber;
    }

    @Override // show the issue number  3 个用法
    void displayInfo() {
        super.displayInfo(); // print title, author, year
        System.out.println("Issue Number: " + issueNumber); // add issue number
    }
}
```

```java
public class LibraryManagementSystem {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        ArrayList<Publication> library = new ArrayList<>(); // store all publications (books and magazines)

        System.out.println("Enter the number of books to add:");
        int numBooks = Integer.parseInt(scanner.nextLine()); // read number of books

        // loop to get details for each book
        for (int i = 1; i <= numBooks; i++) {
            System.out.println("Enter Book " + i + " details:");

            System.out.print("Title: ");
            String title = scanner.nextLine();

            System.out.print("Author: ");
            String author = scanner.nextLine();

            System.out.print("Publication Year: ");
            int year = Integer.parseInt(scanner.nextLine());

            System.out.print("ISBN: ");
            String isbn = scanner.nextLine();

            Book book = new Book(title, author, year, isbn); // create a new Book object with the input info

            library.add(book); // add the book to list
        }

        // get Magazines from user
        System.out.println("Enter the number of magazines to add:");
        int numMagazines = Integer.parseInt(scanner.nextLine()); // read number of magazines

        // loop to get details for each magazine
        for (int i = 1; i <= numMagazines; i++) {
            System.out.println("Enter Magazine " + i + " details:"); // tell user which magazine

            System.out.print("Title: ");
            String title = scanner.nextLine(); // read title

            System.out.print("Author: ");
            String author = scanner.nextLine(); // read author
```

```
128
129             System.out.print("Publication Year: ");
130             int year = Integer.parseInt(scanner.nextLine()); // read year
131
132             System.out.print("Issue Number: ");
133             int issueNum = Integer.parseInt(scanner.nextLine()); // read issue number
134
135             // Create a Magazine object with the info we got
136             Magazine magazine = new Magazine(title, author, year, issueNum);
137
138             library.add(magazine); // add the magazine to our library list
139         }
140
141         System.out.println("\nLibrary Collection:");
142
143         // loop through every item in the library ArrayList
144         for (Publication pub : library) {
145             pub.displayInfo();
146             System.out.println("---");
147         }
148
149         scanner.close();
150     }
151 }
```

Output:

```
Enter the number of books to add:
2
Enter Book 1 details:
Title: Harry Potter
Author: JK
Publication Year: 1996
ISBN: 123213
Enter Book 2 details:
Title: 67
Author: Skim
Publication Year: 2005
ISBN: 23414
Enter the number of magazines to add:
1
Enter Magazine 1 details:
Title: wdsfsdf
Author: XIii
Publication Year: 2025
Issue Number: 12323

Library Collection:
Title: Harry Potter
Author: JK
Publication Year: 1996
ISBN: 123213
```